

Deep Learning

Modelos de Linguagem Decoder-Only

Lucas Silveira Kupssinskü

Machine Learning Theory and Applications Laboratory
PUCRS

maio de 2026

Visão Geral

1. Revisão do Transformer
2. A Família GPT e o Paradigma Decoder-Only
3. Leis de Escala
4. Normalização: Layer Norm e RMS Norm
5. Aproximações de Atenção
6. Mistura de Especialistas (MoE)
7. Estratégias de Decodificação
8. Benchmarks e Avaliação de LLMs

Overview

1. **Revisão do Transformer**
2. A Família GPT e o Paradigma Decoder-Only
3. Leis de Escala
4. Normalização: Layer Norm e RMS Norm
5. Aproximações de Atenção
6. Mistura de Especialistas (MoE)
7. Estratégias de Decodificação
8. Benchmarks e Avaliação de LLMs

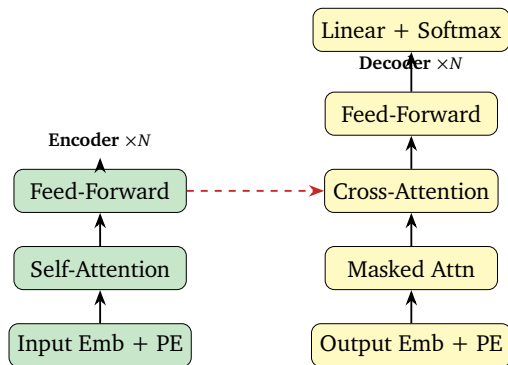
Arquitetura Original: Encoder-Decoder [18]

Encoder (esquerda):

- Lê toda a entrada **bidirecional**
- Self-attention não mascarada
- Produz representações contextuais ricas

Decoder (direita):

- Gera tokens **autorregressivamente**
- Self-attention **mascarada** (causal)
- **Cross-attention**: lê saída do encoder



Pergunta-chave

Precisamos do encoder para geração de linguagem?

Multi-Head Self-Attention

Dado um token, computamos queries, keys e values:

Atenção escalada

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Cada cabeça aprende padrões diferentes:

$$\text{MHA} = \text{Concat}(\text{head}_1, \dots, \text{head}_H) W^O$$

- H cabeças $\times d_k = d_{\text{model}}/H$
- Custo: $O(n^2d)$ por camada
- Paralelizável sobre toda a sequência

Máscara causal (decoder)

Posição i só pode atender a $j \leq i$:

$$M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$$

Bloco Transformer: FFN, Resíduo e Normalização

Cada camada contém:

1. **(Masked) Self-Attention** + resíduo + norma
2. **Feed-Forward** + resíduo + norma

FFN de dois estágios

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

Dimensão interna: $d_{\text{ff}} \approx 4 d_{\text{model}}$

(Valor empírico de [18]; modelos com SwiGLU partem de $\frac{8}{3} d_{\text{model}}$, ajustado para múltiplos convenientes. Para $d = 4096$: 11 008 no Llama 2, 14 336 no Llama 3 (que adota expansão maior).)

Post-Norm (Vaswani 2017)

$$x \leftarrow \text{LN}(x + \text{SubLayer}(x))$$

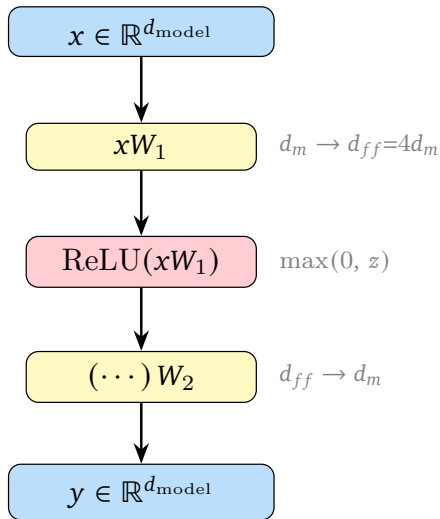
GPT-2 em diante usa **Pre-Norm**:

Pre-Norm (GPT-2+)

$$x \leftarrow x + \text{SubLayer}(\text{Norm}(x))$$

Empilhar N desses blocos =
Transformer.

FFN Padrão: Exemplo Numérico (ReLU)



Exemplo numérico

$$d_{\text{model}} = 3, \quad d_{ff} = 4 \times 3 = 12$$

$$x = [1.0, 0.5, -0.5]$$

$$xW_1 = [2.0, -1.0, 0.5, \dots] \text{ (dim 12)}$$

$$\text{ReLU}(xW_1) = [2.0, 0.0, 0.5, \dots]$$

$\Rightarrow$ ReLU **zera** o valor negativo -1.0 !

$$[\dots]W_2 \approx y = [0.7, -0.2, 0.4]$$

SwiGLU: A FFN com *Gate* das LLMs mais atuais

FFN clássica usa **ReLU**:

ReLU-FFN (Vaswani 2017)

$$\text{FFN}(x) = \max(0, xW_1) W_2$$

Llama 2/3, PaLM, Mistral usam **SwiGLU** [15]:

SwiGLU-FFN

$$\text{FFN}(x) = (\text{SiLU}(xW_1) \odot xW_3) W_2$$

onde $\text{SiLU}(z) = z \cdot \sigma(z)$ (*Swish*).

O **portão** xW_3 filtra dinamicamente a informação.

Por que $\frac{8}{3} d_{\text{model}}$?

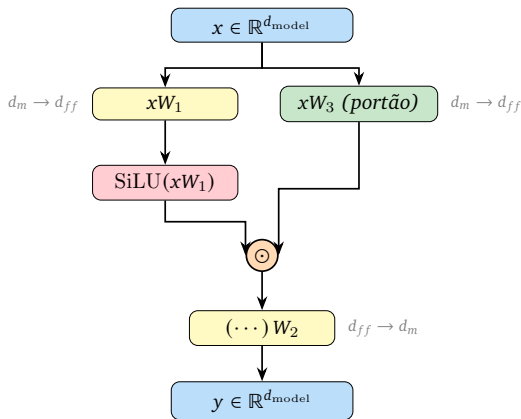
SwiGLU usa **3 matrizes** em vez de 2.
Para manter o mesmo número de parâmetros:

$$3 \times d_{\text{ff}} \cdot d = 2 \times 4d \cdot d$$

$$\Rightarrow d_{\text{ff}} = \frac{8}{3} d_{\text{model}}$$

- SwiGLU supera ReLU e GELU empiricamente
- Motivação teórica: portão aprende *quais* dimensões ativar
- Alternativas: GeGLU, ReGLU

SwiGLU: Diagrama Computacional e Dimensões



Exemplo numérico

$$d_{\text{model}} = 3, \quad d_{ff} = \frac{8}{3} \times 3 = 8$$

$$x = [1.0, 0.5, -0.5]$$

$$xW_1 = [2.0, -1.0, 0.5, 1.2, \dots]$$

$$\text{SiLU}(xW_1) \approx [1.76, -0.27, 0.31, 1.09, \dots]$$

$$xW_3 = [0.8, 0.0, 1.2, 0.5, \dots] \text{ (portão)}$$

$$[1.76, -0.27, 0.31, 1.09, \dots]$$

$$\odot [0.8, 0.0, 1.2, 0.5, \dots]$$

$$= [1.41, 0.00, 0.37, 0.55, \dots]$$

$\Rightarrow$ portão **0.0** suprime a dim. 2!

$$[\dots]W_2 \approx y = [0.7, -0.2, 0.4]$$

O que Encoder e Decoder fazem?

Encoder: compreensão

- Processa **toda a entrada** de uma vez
- Atenção bidirecional
- Produz vetor de contexto rico
- Exemplos: BERT, RoBERTa

Decoder: geração

- Gera **um token por vez**
- Atenção causal (só vê o passado)
- Recebe contexto via cross-attention
- Exemplos: GPT, T5-decoder

Para LLMs, o encoder é desnecessário.

O decoder processa o *prompt* como parte da sequência.

Máscara Causal vs. Atenção Bidirecional

Bidirecional (BERT/Encoder):
cada posição vê *toda* a sequência.

	t_1	t_2	t_3	t_4
t_1	✓	✓	✓	✓
t_2	✓	✓	✓	✓
t_3	✓	✓	✓	✓
t_4	✓	✓	✓	✓

Causal (GPT/Decoder-Only):
posição i só vê $j \leq i$.

	t_1	t_2	t_3	t_4
t_1	✓	×	×	×
t_2	✓	✓	×	×
t_3	✓	✓	✓	×
t_4	✓	✓	✓	✓

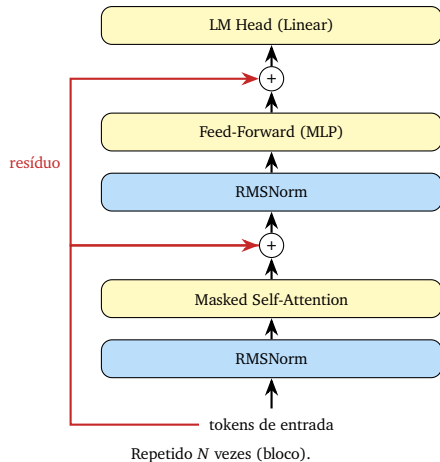
Garante que t_4 não “veja o futuro” durante o treino.

Do Encoder-Decoder para o Decoder-Only

Ideia central: treine apenas o decoder.

O modelo recebe o texto de entrada como *prefixo* e continua gerando autorregressivamente.

- Sem cross-attention
- Sem encoder separado
- Toda informação está nas posições anteriores
- Arquitetura mais simples



Overview

1. Revisão do Transformer
- 2. A Família GPT e o Paradigma Decoder-Only**
3. Leis de Escala
4. Normalização: Layer Norm e RMS Norm
5. Aproximações de Atenção
6. Mistura de Especialistas (MoE)
7. Estratégias de Decodificação
8. Benchmarks e Avaliação de LLMs

GPT-1: Pré-Treino + Fine-Tuning [12]

Radford et al. (2018), OpenAI

Objetivo de pré-treino: modelagem de linguagem causal

$$\mathcal{L} = - \sum_t \log P(x_t \mid x_1, \dots, x_{t-1}; \Theta)$$

Depois: *fine-tuning* supervisionado para cada tarefa.

- 117M parâmetros
- 12 camadas, $d = 768$, 12 cabeças
- BooksCorpus (≈ 800 M tokens)
- Tokenização BPE

Contribuição

Mostrou que pré-treino auto-supervisionado em texto melhora fortemente o fine-tuning.

GPT-2: Geração Zero-Shot [13]

Radford et al. (2019)

Hipótese: um LM suficientemente grande aprende a realizar tarefas **sem fine-tuning**.

“Language models are unsupervised multitask learners”

Novidades técnicas:

- Pre-Norm ao invés de Post-Norm
- Contexto: 1024 tokens
- Vocabulário: 50257 tokens (BPE^a)

^aBPE = *Byte-Pair Encoding*: segmenta palavras em sub-palavras frequentes, reduzindo o tamanho do vocabulário.

Versão	Params	Camadas	d	H
Small	117M	12	768	12
Medium	345M	24	1024	16
Large	762M	36	1280	20
XL	1.5B	48	1600	25

Dataset: WebText (40GB de texto web filtrado).

GPT-3: Zero e Few-shot learning[3]

Brown et al. (2020), 175 bilhões de parâmetros

Few-shot prompting: exemplos fornecidos *no contexto*, sem atualização de pesos.

Modos de uso em contexto

- **Zero-shot:** só a instrução
- **One-shot:** 1 exemplo no prompt
- **Few-shot:** K exemplos no prompt

- 96 camadas, $d = 12288$, 96 cabeças
- Contexto: 2048 tokens
- Treinado em 300B tokens
- **Capacidades emergentes:** habilidades que *surgem abruptamente* com escala (aritmética, tradução, raciocínio); ausentes em modelos menores

Mudança de paradigma

A partir do GPT-3, o fine-tuning por tarefa foi substituído pela engenharia de *prompts*.

Bloco Transformer Decoder-Only em Detalhe

O bloco básico repete-se N vezes:

1. **Normalização** (Pre-Norm)
2. **Masked Self-Attention**
3. Conexão residual: $x += \text{Attn}(\cdot)$
4. **Normalização**
5. **Feed-Forward** (MLP)
6. Conexão residual: $x += \text{FFN}(\cdot)$

Sem cross-attention. Toda informação está nas n posições anteriores da sequência.

KV-Cache

Durante inferência, K e V já calculados são armazenados.

Sem cache: custo $O(n^2d)$ por token

Com cache: custo $O(nd)$ por token

Essencial para geração eficiente em sequências longas.

KV-Cache: Como Funciona na Prática

Gerando a sequência “O rato roeu a rolha”:

1. **Prefill:** alimenta o *prompt* inteiro de uma vez
→ computa e *armazena* K, V de cada token
2. **Decode passo 1:** prediz próximo token
→ computa K, V só do token novo
→ reutiliza K, V dos tokens anteriores do cache
3. **Decode passo 2, 3, ...:** idem
→ cache cresce a cada passo gerado

Sem cache (naïve)

Para o n -ésimo token:

Recalcula K, V de todos os n tokens

Custo total: $O(n^2d)$

Com cache

Computa K, V apenas do token novo

Concatena com o cache existente

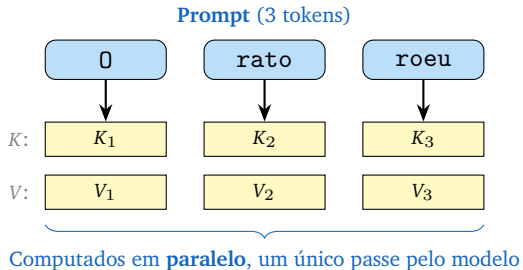
Custo por token: $O(nd)$

Memória do cache $\propto n \cdot L \cdot d_k$, cresce com a sequência e número de camadas.

KV-Cache: Visualização

Prefill

Exemplo: “O rato roeu a rolha”, todos os K,V do prompt são computados de uma vez.

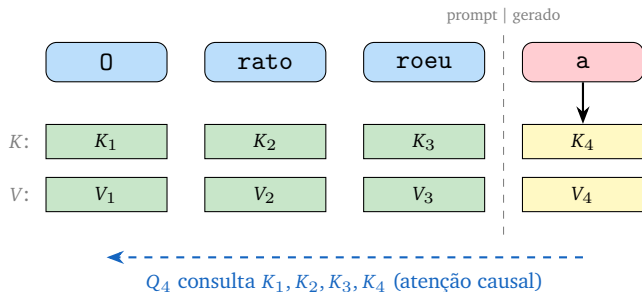


■ = token ■ = computando agora

KV-Cache: Visualização

Decode (Passo 1)

Gerando "a": apenas K_4, V_4 são computados; K_1, K_2, K_3 vêm do cache.

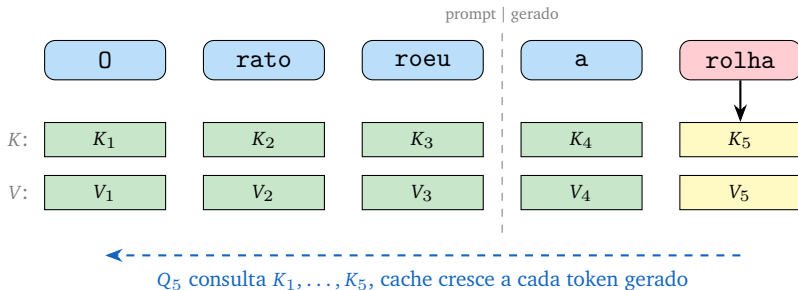


■ = cache (reutilizado) ■ = computando agora

KV-Cache: Visualização

Decode (Passo 2)

Gerando "rolha": apenas K_5, V_5 são computados; cache cresce para 4 entradas.



■ = cache (reutilizado) ■ = computando agora

Overview

1. Revisão do Transformer
2. A Família GPT e o Paradigma Decoder-Only
- 3. Leis de Escala**
4. Normalização: Layer Norm e RMS Norm
5. Aproximações de Atenção
6. Mistura de Especialistas (MoE)
7. Estratégias de Decodificação
8. Benchmarks e Avaliação de LLMs

O que Significa “Escalar” um LLM?

Três eixos de escala:

1. **Parâmetros** N : tamanho do modelo
2. **Dados** D : tokens de treinamento
3. **Compute** C : FLOPs totais de treino

(FLOP = *Floating Point Operation*, unidade de custo aritmético)

Relação aproximada:

$$C \approx 6ND$$

Cada token exige ≈ 6 FLOPs por parâmetro:
2 no *forward* + 4 no *backward* [10].

Pergunta central

Dado um orçamento de compute fixo, como distribuir entre N (modelo maior) e D (mais dados)?

Isso é exatamente o que as **leis de escala** respondem.

Leis de Escala de Kaplan et al. [10]

Descoberta: a perda de teste segue **leis de potência**:

$$L(N) \propto N^{-\alpha_N}, \quad L(D) \propto D^{-\alpha_D}$$

com $\alpha_N \approx 0.076$, $\alpha_D \approx 0.095$.

Recomendação: dado compute fixo, priorize **escalar** N e use menos dados.

- Válido para modelos de 768M a 1.5B parâmetros
- Curvas de perda suaves e previsíveis
- Arquitetura (profundidade vs. largura) tem impacto menor

Limitação

Kaplan et al. não otimizavam N e D simultaneamente; fixavam N e mediam L ótima em C .

Chinchilla: Treinamento Compute-Ótimo [7]

Hoffmann et al. (2022), DeepMind
Treinaram 400+ modelos variando N e D
com mesmo C .

Resultado central

Para compute ótimo:

$$N^* \propto C^{0.5}, \quad D^* \propto C^{0.5}$$

Dobrar compute $\rightarrow$ dobrar N E dobrar D .

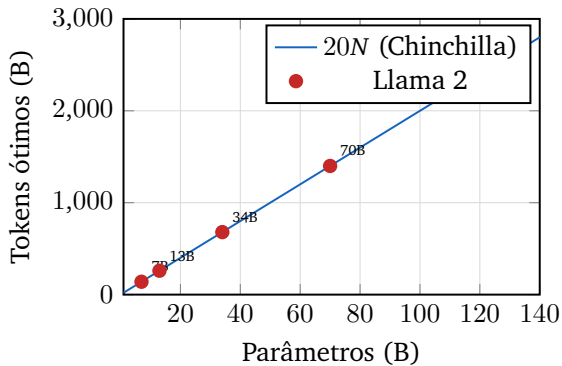
Chinchilla (70B) $\approx$ Gopher (280B)
em qualidade, com 4 $\times$ menos
compute de inferência.

Regra prática

Para um modelo de N parâmetros,
treine com pelo menos $20N$ **tokens**.

Exemplo: modelo de 7B $\rightarrow \geq 140$ B
tokens.

Relação Ótima Tokens/Parâmetros



- Linha azul: regra dos 20N tokens (Chinchilla)
- Modelos recentes usam **muito mais dados**: Chinchilla sugere 20N para 8B = 160B tokens; Llama 3 usou 15T, ~93× mais

Compute-ótimo $\neq$ inferência-ótimo

Modelos menores treinados por mais tempo custam menos na inferência, preferíveis quando há muitas chamadas.

Implicações Práticas das Leis de Escala

- **Llama 1** (Meta, 2023): 7B–65B treinados com 1T–1.4T tokens
- **Llama 2** (Meta, 2023): até 2T tokens, contexto 4096
- **Llama 3** (Meta, 2024): 8B e 70B com 15T tokens
- **Mistral 7B** (2023): supera Llama 2 13B com 7B parâmetros

Lição aprendida

Treinar modelos **menores por mais tempo** é mais custo-efetivo para implantação do que modelos gigantes subtreinados.

A Chinchilla reorientou o campo: de corrida por parâmetros (GPT-3, Gopher, Megatron) para equilíbrio $N \times D$.

Modelos Recentes: Parâmetros, Dados e Contexto

Modelo	Params	Tokens treino	Contexto	Norm	Atenção
GPT-2 XL	1.5B	~40B	1 024	Pre-LN	MHA
GPT-3	175B	300B	2 048	Pre-LN	MHA
Llama 2 7B	7B	2T	4 096	RMSNorm	GQA
Llama 2 70B	70B	2T	4 096	RMSNorm	GQA
Llama 3 8B	8B	15T	8 192	RMSNorm	GQA
Mistral 7B	7B	~1T	32 768	RMSNorm	GQA+SWA
Mixtral 8×7B	46.7B	~1T	32 768	RMSNorm	GQA+MoE

SWA = Sliding Window Attention; MoE = Mistura de Especialistas

Overview

1. Revisão do Transformer
2. A Família GPT e o Paradigma Decoder-Only
3. Leis de Escala
- 4. Normalização: Layer Norm e RMS Norm**
5. Aproximações de Atenção
6. Mistura de Especialistas (MoE)
7. Estratégias de Decodificação
8. Benchmarks e Avaliação de LLMs

Por que Normalizar as Ativações?

Durante o treino de redes profundas:

- Distribuição das ativações muda a cada camada (*internal covariate shift*)
- Gradientes explodem ou desaparecem
- Taxas de aprendizado devem ser muito pequenas

Batch Norm: normaliza sobre o batch, problemático para sequências de tamanho variável.

Solução para Transformers

Layer Normalization [2]: normaliza sobre as *features* de cada token, sem dependência do batch.

Permite treino estável de Transformers muito profundos com learning rates maiores.

Layer Normalization [2]

Dado o vetor de ativações $\mathbf{x} \in \mathbb{R}^d$ de um token:

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$$

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$\text{LN}(\mathbf{x}) = \boldsymbol{\gamma} \odot \hat{\mathbf{x}} + \boldsymbol{\beta}$$

$\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$: parâmetros aprendidos (*gain* e *bias*).

- Independente do tamanho do batch
- Funciona com sequências de tamanho variável
- Aplicada **por token** (ao longo da dimensão d)

Nota

A normalização ocorre ao longo da dimensão de features d de cada token, não ao longo da sequência.

Pre-Norm vs. Post-Norm

Post-Norm (Vaswani 2017)

$$x \leftarrow \text{LN}(x + \text{SubLayer}(x))$$

Normalização **depois** da operação.

→ gradientes instáveis com N grande;
requer *warm-up* lento.

Pre-Norm (GPT-2 em diante)

$$x \leftarrow x + \text{SubLayer}(\text{LN}(x))$$

Normalização **antes** da operação.

→ treino mais estável; gradientes fluem
melhor pela conexão residual.

Hoje, quase todos os LLMs usam **Pre-Norm**.

A conexão residual “bypassa” a normalização, preservando o sinal ao longo das camadas.

RMS Norm: Simplificação do Layer Norm [21]

Zhang & Sennrich (2019): o re-centramento (subtração de μ) é desnecessário quando a invariância de escala é suficiente.

Normaliza apenas pela raiz do valor quadrático médio:

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2}$$

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \boldsymbol{\gamma}$$

Sem μ , sem β , mais simples e rápido.

- Mais rápido: sem cálculo de média
- Mesma qualidade empírica que LayerNorm
- Menos parâmetros (sem β)

Adotado por

Llama 1/2/3, Mistral, Mixtral, Gemma, Falcon, PaLM, ...

Citado em [17] como padrão atual para LLMs eficientes.

Qual Normalização Cada Modelo Usa?

Modelo	Norma	Posição	Bias β ?	Observação
BERT	LayerNorm	Post-Norm	Sim	Encoder bidirecional
GPT-2	LayerNorm	Pre-Norm	Sim	Decoder-only
GPT-3	LayerNorm	Pre-Norm	Sim	Decoder-only
Llama 1/2/3	RMSNorm	Pre-Norm	Não	Decoder-only
Mistral 7B	RMSNorm	Pre-Norm	Não	Decoder-only
Gemma	RMSNorm	Pre+Post	Não	“Sandwich” norm, estabiliza treino profundo
PaLM	RMSNorm	Pre-Norm	Não	Decoder-only

Tendência: RMSNorm + Pre-Norm para LLMs modernos.

Overview

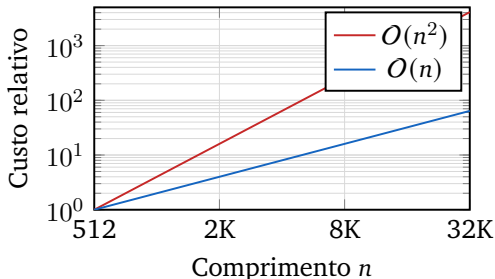
1. Revisão do Transformer
2. A Família GPT e o Paradigma Decoder-Only
3. Leis de Escala
4. Normalização: Layer Norm e RMS Norm
- 5. Aproximações de Atenção**
6. Mistura de Especialistas (MoE)
7. Estratégias de Decodificação
8. Benchmarks e Avaliação de LLMs

O Gargalo Quadrático da Atenção

Custo da atenção padrão [18]:

- Memória: $O(n^2)$, matriz de scores
- Compute: $O(n^2d)$, produto QK^T

Com $n = 32\,768$ tokens e $H = 32$ cabeças:
Matriz de atenção: $32768^2 \times 32 \approx 34$ GB
(float16)



Motivação: reduzir custo ao escalar o contexto [17].

Multi-Head Attention (MHA)

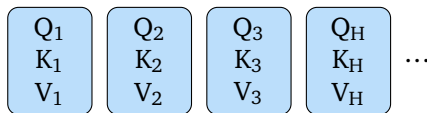
Recapitulação

Com H cabeças, $d_k = d_{\text{model}}/H$:

- H projeções de Q, K, V independentes
- H pares de matrizes KV em cache por camada
- Custo de KV-cache por token: $2 \cdot H \cdot d_k \cdot L$

Para GPT-3 ($H = 96$, $d_k = 128$, $L = 96$):

Cache = $2 \times 96 \times 128 \times 96 \approx 2.4\text{M floats/token}$



KV-cache: H pares por camada

Gargalo em inferência: KV-cache cresce com n , H , L .

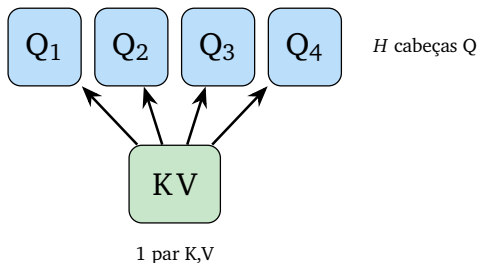
Multi-Query Attention (MQA) [14]

Shazeer (2019): reduza K e V para **uma única cabeça** compartilhada; mantenha H cabeças de Q.

$$\text{head}_h = \text{Attn}(QW_h^Q, KW^K, VW^V)$$

Redução de cache: $H \times \rightarrow 1 \times$ nos tensores K, V.

Cache por token: $2 \cdot d_k \cdot L$ (independente de H).



- KV-cache $H \times$ menor
- Inferência muito mais rápida
- Leve queda de qualidade

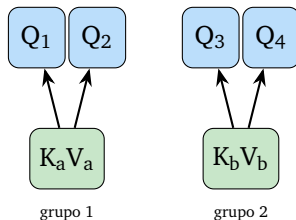
Grouped Query Attention (GQA) [1]

Ainslie et al. (2023): compromisso entre MHA e MQA.

Divida as H cabeças de Q em G grupos. Cada grupo compartilha um par K,V.

$$G = 1 \Rightarrow \text{MQA} \quad G = H \Rightarrow \text{MHA}$$

Llama 2/3, Mistral, Mixtral usam GQA.



- $G = 8$ típico (Llama 2 70B: $H = 64, G = 8$)
- Cache reduzido $H/G \times$
- Qualidade quase idêntica ao MHA

Comparação: MHA, MQA e GQA

Variante	Pares K,V	Cache KV	Qualidade	Exemplos
MHA	H	$H \cdot d_k$	Referência	GPT-2, GPT-3, BERT
MQA	1	d_k	↓ levemente	PaLM, Falcon
GQA	G ($1 < G < H$)	$G \cdot d_k$	$\approx$ MHA	Llama 2/3, Mistral

GQA na prática

Llama 2 70B: $H = 64$ cabeças Q, $G = 8$ grupos KV.
Cache 8× menor que MHA com qualidade semelhante.

Por que importa?

Redução do KV-cache é fundamental para servir muitos usuários simultaneamente, o cache de todos os requests cabe na GPU.

Sliding Window Attention [8]

Motivação: em sequências longas, a maioria das informações úteis está **próxima** do token atual.

Cada posição i atende apenas às W posições anteriores:

$$\text{Attn}(i) \leftarrow \text{tokens em } [i - W, i]$$

Custo: $O(n \cdot W)$ em vez de $O(n^2)$.

Campo receptivo em L camadas: $L \cdot W$.

Informação distante propaga-se por composição: camada l agrega o que camada

$l-1$ já resumiu da janela anterior.

Configuração Mistral 7B

- Janela local: $W = 4096$
- 32 camadas
- Campo receptivo:
 $32 \times 4096 = 131K$
- Contexto declarado: 32K tokens

Resultado: Mistral 7B supera Llama 2 13B em benchmarks com 2× menos parâmetros.

Resumo: Variantes de Atenção e Trade-offs

Variante	Compute	Mem. KV	Contexto longo	Qualidade	Exemplos
MHA (padrão)	$O(n^2d)$	$H \cdot d_k$	Limitado	Referência	GPT-2/3
MQA	$O(n^2d)$	d_k	Moderado	↓ levemente	PaLM, Falcon
GQA	$O(n^2d)$	$G \cdot d_k$	Moderado	$\approx$ MHA	Llama 2/3, Mistral
SWA	$O(nWd)$	$W \cdot d_k$	Alto	Boa (local)	Mistral

Baseado em [17]. SWA = Sliding Window Attention.

Overview

1. Revisão do Transformer
2. A Família GPT e o Paradigma Decoder-Only
3. Leis de Escala
4. Normalização: Layer Norm e RMS Norm
5. Aproximações de Atenção
- 6. Mistura de Especialistas (MoE)**
7. Estratégias de Decodificação
8. Benchmarks e Avaliação de LLMs

Motivação: Mais Parâmetros sem Mais Compute

Dilema:

- Modelos maiores $\rightarrow$ melhor qualidade
- Modelos maiores $\rightarrow$ mais compute por token

Ideia da Mistura de Especialistas (MoE):

Substitua a FFN densa por E redes menores (*especialistas*).

Cada token ativa apenas k especialistas ($k \ll E$).

Exemplo: Mixtral $8 \times 7B$

8 especialistas FFN

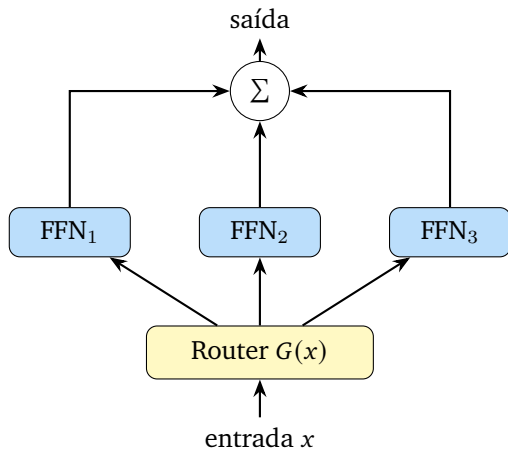
Apenas 2 ativados por token

$\rightarrow$ 46.7B parâmetros totais

$\rightarrow$ compute $\approx$ 12.9B por token

Origem: **Sparsely-Gated MoE** [16] (2017), revivido em LLMs a partir de 2022.

Arquitetura MoE: Router + Especialistas



Em cada bloco MoE:

1. Router calcula scores para cada especialista
2. Seleciona top- k especialistas
3. Processa x nos k especialistas
4. Combina saídas ponderadas pelos scores

Nota importante

As camadas de **atenção** permanecem densas.
Somente a FFN vira MoE.

Top- k Routing: Mecanismo de Gating

Dado token $x \in \mathbb{R}^d$ e pesos $W_g \in \mathbb{R}^{d \times E}$:

$$g(x) = \text{softmax}(\text{TopK}(xW_g, k))$$

onde TopK mantém os k maiores scores e zera o resto.

Saída:

$$y = \sum_{i \in \text{Top-}k} g_i(x) \text{FFN}_i(x)$$

- Mixtral usa $k = 2$, $E = 8$
- ~25% dos parâmetros ativos por token
- Backward: apenas pelos k especialistas ativos

Expert collapse

Causa: o router aprende a enviar tokens ao especialista que já é o melhor, que melhora ainda mais, atraindo mais tokens, em ciclo de reforço positivo.

Resultado: 1–2 especialistas processam quase tudo; os demais recebem gradiente zero e nunca aprendem.

Solução: *Loss* auxiliar de balanceamento (próximo slide).

Problemas de MoE: Balanceamento de Carga

Perda auxiliar de balanceamento:

$$\mathcal{L}_{\text{aux}} = \alpha \cdot E \cdot \sum_{i=1}^E f_i \cdot P_i$$

onde f_i = fração de tokens roteados ao especialista i ,

P_i = score médio do router para o especialista i .

Minimizar $\mathcal{L}_{\text{aux}}$ encoraja distribuição uniforme.

Outros desafios:

- **Instabilidade:** gradientes esparsas podem causar divergência
- **Comunicação:** especialistas distribuídos em GPUs diferentes (*model parallelism*)
- **Capacidade máxima:** tokens descartados se um especialista ficar sobrecarregado

Mixtral

Usa $\alpha = 0.01$, roteamento token a token.

Mixtral 8×7B [9]

Jiang et al. (2024), Mistral AI

Arquitetura:

- 32 camadas transformer
- Cada FFN → 8 especialistas de 14336 neurônios
- Top-2 routing por token
- GQA ($G = 8$), RMSNorm, SWA
- Contexto: 32 768 tokens

Modelo	Params ativos	Params totais
Llama 2 13B	13B	13B
Llama 2 70B	70B	70B
Mistral 7B	7B	7B
Mixtral 8×7B	12.9B	46.7B

Resultado

Mixtral 8×7B supera Llama 2 70B em benchmarks com 5.4× menos parâmetros ativos.

MoE Sparse vs. Modelos Densos

Característica	Denso (Llama 2 70B)	Sparse MoE (Mixtral 8×7B)
Parâmetros totais	70B	46.7B
Parâmetros ativos	70B	12.9B
Compute por token	Alto	~ 5× menor
Memória total (fp16)	~140 GB	~93 GB
Throughput (tokens/s)	Menor	Maior
Qualidade (MMLU)	68.9%	70.6%

MoE: mais capacidade com menor custo de inferência,
ao custo de maior memória total e complexidade de treino.

Overview

1. Revisão do Transformer
2. A Família GPT e o Paradigma Decoder-Only
3. Leis de Escala
4. Normalização: Layer Norm e RMS Norm
5. Aproximações de Atenção
6. Mistura de Especialistas (MoE)
- 7. Estratégias de Decodificação**
8. Benchmarks e Avaliação de LLMs

Estratégias de Decodificação: Greedy, Beam, Sampling

Após o LM produzir logits $z \in \mathbb{R}^{|V|}$:

Greedy

$$x_t = \arg \max_v P(v \mid x_{<t})$$

Rápido, determinístico, pode ser subótimo.

Beam Search

Mantém B hipóteses; escolhe a de maior probabilidade conjunta. $O(B)$ mais lento; ótimo para tradução.

Amostragem

$$x_t \sim P(\cdot \mid x_{<t})$$

Estocástico; necessário para criatividade.

Parâmetros de controle:

- **Temperature** T : $P_T \propto z_v/T$
- **Top-k**: restringe ao top-k tokens
- **Top-p** (nucleus): restringe ao menor conjunto com prob. acumulada $\geq p$

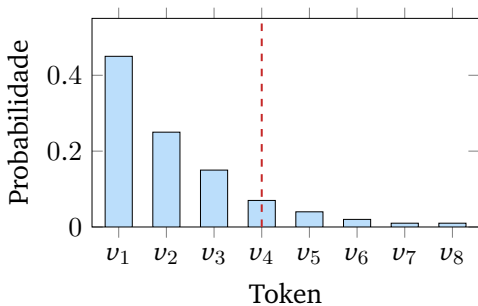
Temperature, Top- k e Top- p

Temperature:

$$P_T(x) = \frac{\exp(z_x/T)}{\sum_v \exp(z_v/T)}$$

- $T \rightarrow 0$: greedy
- $T = 1$: distribuição original
- $T > 1$: distribuição mais uniforme

Top- p : selecione o menor conjunto de tokens cuja probabilidade acumulada $\geq p$ (ex: $p = 0.9$), então amostra dentro desse conjunto.



Decodificação Guiada (*Constrained Decoding*) [19]

Problema: LLMs podem gerar texto que não segue formato específico (JSON, SQL, código).

Solução: restringir a distribuição de saída a tokens válidos segundo uma **gramática** ou **esquema**.

Willard & Louf (2023): pré-compila um DFA para cada gramática, o mascaramento de tokens inválidos é $O(1)$ por token.

Exemplos de uso

- Geração de JSON estruturado
- SQL sem erros de sintaxe
- Extração em formato fixo
- Chamadas de API com parâmetros corretos

Implementações: outlines, guidance, llama.cpp grammar mode, vLLM structured outputs.

Decodificação Especulativa: Ideia Geral [11]

Gargalo: LLMs grandes são lentos, cada token exige um passe completo.

Observação: a maioria dos tokens é fácil de prever; tokens difíceis são exceção.

Algoritmo:

1. **Draft model** pequeno gera y tokens
2. **Target model** verifica *em paralelo*
3. Tokens aceitos $\rightarrow$ mantidos; primeiro rejeitado $\rightarrow$ re-amostrado

Propriedade chave

Saída **idêntica em distribuição** ao modelo alvo, sem perda de qualidade por construção.

Speedup típico

2–3 $\times$ com draft 10 $\times$ menor.
Ex: Llama 3 70B + Llama 3 8B como draft.

Algoritmo Especulativo: Critério de Aceitação

Draft model M_q , target model M_p :

Para cada token $\tilde{x}_t$ proposto pelo draft:

$$\text{Aceitar com prob. } \min\left(1, \frac{p(\tilde{x}_t | x_{<t})}{q(\tilde{x}_t | x_{<t})}\right)$$

Se rejeitado: re-amostre de

$$p'(x) \propto \max(0, p(x) - q(x))$$

Verifique todos os γ tokens **em um único passe** do modelo alvo.

- $p \geq q$: token **sempre aceito**
- Draft e target concordam $\rightarrow$ aceleração máxima
- Tokens difíceis: target corrige

Complexidade

$\gamma + 1$ passos de M_q + 1 passe de M_p
 $\rightarrow$ até $\gamma + 1$ tokens gerados.

Sem especulação: $\gamma + 1$ passes de M_p .

Decodificação Especulativa: Exemplo Passo a Passo

Contexto: completar “O gato subiu na ____” com $\gamma = 3$.

Draft M_q propõe: árvore, e, dormiu, verificados em **um único passe** de M_p .

Token	q	p	$\min\left(1, \frac{p}{q}\right)$	Decisão
árvore	0,6	0,9	1,00	sempre aceito
e	0,7	0,4	0,57	aceito c/ prob. 57%
dormiu	0,8	0,2	0,25	aceito c/ prob. 25%

Cenário A (todos aceitos)

3 tokens em 1 passe de M_p .
Sem especulação: 3 passes.

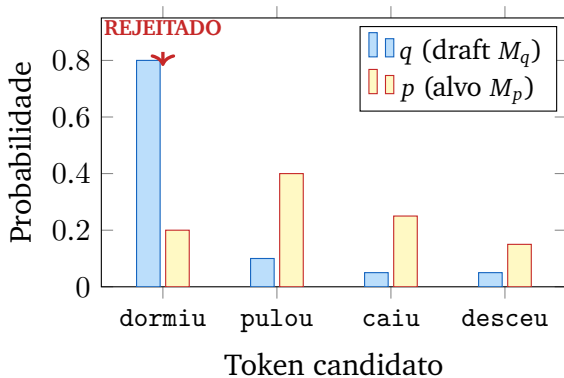
Cenário B (e rejeitado)

Re-amostra de $p' \propto \max(0, p-q)$;
somente **árvore** confirmado,
ainda 1 passe.

Rejeição: Distribuições q e p para dormiu

Contexto: “O gato subiu na árvore e ____” (árvore e e já aceitos).

Draft M_q propõe dormiu com $q=0,8$; modelo alvo tem $p=0,2$.



Por que dormiu é rejeitado?

$$p(\text{dormiu}) = 0,2 < q(\text{dormiu}) = 0,8$$

Aceitar com probabilidade:

$$\min\left(1, \frac{p}{q}\right) = \frac{0,2}{0,8} = 0,25$$

Neste cenário, **dormiu é rejeitado**.

Observação

M_p prefere pulou ($p=0,4$), o draft superestimou a certeza em dormiu.

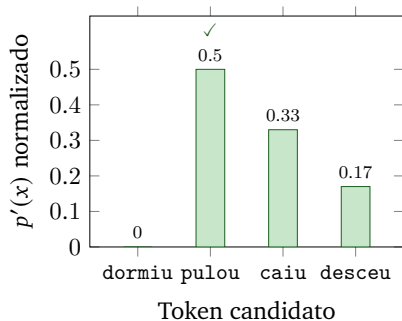
Re-amostragem: $p'(x) \propto \max(0, p(x) - q(x))$

Token	p	q	$p-q$	p' (norm.)
dormiu	0,20	0,80	-0,60	0,00
pulou	0,40	0,10	+0,30	0,50
caiu	0,25	0,05	+0,20	0,33
desceu	0,15	0,05	+0,10	0,17

Token selecionado de p'

pulou ($p'=0,50$)

“O gato subiu na árvore e pulou...”



Garantia de correção

p' concentra massa onde M_p discorda de M_q , garantindo distribuição **idêntica a amostrar de M_p** .

Decodificação Especulativa: Variantes e Resultados

Leviathan et al. (2023):

- Testado: T5-XXL (11B) + T5-small (60M) como draft
- Speedup 2–3× dependendo da tarefa
- HumanEval (código): maior speedup (tokens repetitivos)
- Sem degradação de qualidade

Variantes modernas:

- **Self-speculative**: camadas rascunho do próprio modelo
- **Medusa**: múltiplas cabeças de predição em paralelo
- **Eagle**: alinha representações de draft e target

Requisito

Benefício depende da taxa de aceitação α : quanto mais similares os modelos, maior α e maior o speedup.

Overview

1. Revisão do Transformer
2. A Família GPT e o Paradigma Decoder-Only
3. Leis de Escala
4. Normalização: Layer Norm e RMS Norm
5. Aproximações de Atenção
6. Mistura de Especialistas (MoE)
7. Estratégias de Decodificação
- 8. Benchmarks e Avaliação de LLMs**

Perplexidade: A Métrica Mais Simples

Definição: mede o quão “surpreso” o modelo fica ao ver o texto.

$$\text{PPL}(w_1, \dots, w_N) = \exp\left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid w_1, \dots, w_{i-1})\right)$$

- Menor PPL $\rightarrow$ modelo estima melhor a distribuição do corpus
- PPL = 1: certeza absoluta; PPL = $|V|$: uniforme sobre o vocabulário
- Diretamente ligada à *cross-entropy loss* do treino

Intuição

PPL = 20 $\Rightarrow$ o modelo hesita, em média, entre 20 tokens igualmente prováveis a cada passo.

Perplexidade: Por que Não é Suficiente?

Limitações da perplexidade como única métrica

- **Acurácia factual:** um modelo com PPL baixa pode ainda assim alucinar fatos
- **Raciocínio:** baixa surpresa no texto não implica capacidade de resolver problemas
- **Seguimento de instruções:** não captura se o modelo obedece comandos do usuário
- **Desempenho em tarefas reais:** PPL não prediz bem resultados em downstream tasks

⇒ Precisamos de benchmarks específicos por tarefa para avaliar LLMs adequadamente.

Desafios na Avaliação de LLMs

Por que avaliar LLMs é difícil?

- Saída livre → métricas automáticas são imperfeitas
- *Data contamination*: benchmarks no pré-treino
- Capacidades emergentes: falha em instâncias simples, acerta difíceis
- Benchmarks ficam saturados rapidamente

Categorias de avaliação

- **Múltipla escolha**: ARC, MMLU
- **Geração + verificação**: GSM8k
- **Raciocínio**: HellaSwag, WinoGrande
- **Instrução**: MT-Bench, AlpacaEval
- **Arena humana**: Chatbot Arena (Elo)

ARC: AI2 Reasoning Challenge [4]

Clark et al. (2018), Allen Institute for AI
7787 questões de múltipla escolha de
ciências (ensino fundamental/médio).

Duas partições:

- **ARC-Easy:** respondidas por sistemas de recuperação simples
- **ARC-Challenge:** exigem raciocínio; falham em sistemas simples

Exemplo (ARC-Challenge):

“Qual propriedade de um asteróide seria afetada de forma diferente pela gravidade de Júpiter e da Terra?”

(A) cor (B) massa (C) volume (D) **peso**

Avaliação: 0-shot ou 25-shot.

Resultados recentes

Llama 3 8B: 79.7% (ARC-C 25-shot)

GPT-4: $\approx 96\%$

GSM8k: Raciocínio Matemático [5]

Cobbe et al. (2021), OpenAI

8500 problemas matemáticos de nível fundamental.

Características:

- Resolução em múltiplos passos (2–8 operações)
- Resposta final é um número inteiro
- Avaliado com *chain-of-thought* (CoT), o

modelo mostra o raciocínio passo a passo antes da resposta final

Exemplo:

“Natalia vendeu 48 clips em abril e metade disso em maio. Total?” $\Rightarrow 48 + 24 = 72$

Modelo	GSM8k (%)
GPT-2 (1.5B)	≈ 0
GPT-3 (175B)	17.9
Llama 2 7B	14.6
Llama 2 70B	56.8
Mixtral 8×7B	74.4
Llama 3 8B	79.6
Llama 3 70B	93.0
GPT-4	92.0

MMLU: Massive Multitask Language Understanding [6]

Hendrycks et al. (2021), UC Berkeley

15 908 questões em **57 áreas**:

- Matemática, física, química, biologia
- Direito, medicina, economia
- História, filosofia, ciência da computação

Nível: colegial ao profissional avançado.

Avaliação: 5-shot.

Modelo	MMLU (%)
GPT-3 (175B)	43.9
Llama 2 7B	45.3
Llama 2 70B	68.9
Mixtral 8×7B	70.6
Llama 3 8B	66.6
Llama 3 70B	79.5
GPT-4	86.4
Humano (estimado)	≈89.8

Exemplo: Conhecimento Clínico

Qual achado laboratorial é mais típico de anemia ferropriva?

(A) MCV elevado (B) Ferritina elevada

(C) MCV reduzido e RDW elevado (D) B12 reduzida

HellaSwag e WinoGrande: Raciocínio de Senso Comum

HellaSwag [20]

Zellers et al. (2019)

Completar a continuação mais plausível.

“Alguém lava um carro. Ele esfrega o capô...”

(A) e joga fora.

(B) por muito tempo.

(C) com esponja verde.

(D) e come um bolo.

GPT-3: 78.9%; Llama 3 8B: 82.0%

WinoGrande

Sakaguchi et al. (2021)

Resolução de co-referência adversarial.

“Mark é mais alto que Paul, então ele agachou na foto.”

(Mark ou Paul agachou?)

Requer raciocínio físico/social.

Llama 3 8B: 73.0%; GPT-4: $\approx 87\%$

Open LLM Leaderboard: Como Comparar Modelos

Hugging Face mantém *leaderboard* público de LLMs avaliados em condições padronizadas:

- ARC-Challenge (25-shot)
- HellaSwag (10-shot)
- MMLU (5-shot)
- TruthfulQA (0-shot)
- WinoGrande (5-shot)
- GSM8k (5-shot)

Média dos 6 = **score composto**.

Cuidados na interpretação

- Modelos treinados para benchmarks
- Contaminação de dados de teste
- Prompts e few-shot afetam resultados
- Saturação rápida (MMLU-Pro, ARC-AGI)

Comparativo de Modelos nos Principais Benchmarks

Modelo	ARC-C	HSwag	MMLU	TruthQA	WinoG	GSM8k	Média
Llama 2 7B	53.1	78.6	45.3	39.4	74.0	14.6	50.8
Llama 2 13B	59.4	82.1	54.8	37.4	77.0	28.7	56.6
Llama 2 70B	67.3	87.3	68.9	44.9	83.7	56.8	68.2
Mistral 7B	60.0	81.3	60.1	42.2	75.3	52.2	61.8
Mixtral 8×7B	66.4	86.7	70.6	46.8	81.2	74.4	71.0
Llama 3 8B	79.7	82.0	66.6	44.0	73.0	79.6	70.8
Llama 3 70B	93.0	92.2	79.5	52.8	85.3	93.0	82.6

Valores aproximados do Open LLM Leaderboard (2024). HSwag = HellaSwag; TruthQA = TruthfulQA; WinoG = WinoGrande.

Benchmarks Específicos por Tarefa

Benchmarks gerais (MMLU, ARC...) medem conhecimento amplo, mas **não capturam** capacidades especializadas:

Contexto longo

- **RULER**: tarefas sintéticas em janelas de 4K–128K tokens
- **NIAH**: “agulha” escondida em textos de 100K+ tokens

Chamada de ferramentas

- **BFCL**: 2000 pares função/argumento em múltiplas linguagens
- **ToolBench**: uso de APIs reais via ReAct

Código

- **HumanEval**: geração de código Python; métrica *pass@k*
- **SWE-bench**: resolve issues reais do GitHub

Outros domínios

- **MT-Bench**: instrução multi-turno avaliada por GPT-4
- **MATH**: 12500 problemas de competição
- **MMMU**: multimodal (imagem + texto), 57 disciplinas

⇒ Nenhum benchmark único é suficiente: avaliação requer um portfólio de tarefas.

Resumo da Aula

Tópico	Conceito-chave	Inovação / Referência	Modelo exemplo
Decoder-only	Atenção causal, sem encoder	GPT [12]	GPT, Llama
Leis de escala	$20N$ tokens; $N^* \propto C^{0.5}$	Chinchilla [7]	Llama 3
Normalização	RMSNorm + Pre-Norm	[2, 21]	Llama 2/3
MHA $\rightarrow$ GQA	Redução de KV-cache	[1]	Llama 2 70B
Sliding Window	Atenção local $O(nW)$	[8]	Mistral 7B
MoE	Top- k routing, especialistas	[16, 9]	Mixtral $8 \times 7B$
Decod. especulativa	Draft + verify	[11]	Llama 3
Benchmarks	ARC, GSM8k, MMLU, HellaSwag	[4, 5, 6]	—

Referências I

- [1] Joshua Ainslie et al. “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”. Em: [EMNLP](#). 2023, pp. 4895–4901.
- [2] Jimmy Lei Ba, Jamie Ryan Kiros e Geoffrey E. Hinton. “Layer Normalization”. Em: [arXiv preprint arXiv:1607.06450](#) (2016).
- [3] Tom B. Brown et al. “Language Models are Few-Shot Learners”. Em: [NeurIPS](#). Vol. 33. 2020, pp. 1877–1901.
- [4] Peter Clark et al. “Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge”. Em: [arXiv preprint arXiv:1803.05457](#) (2018).
- [5] Karl Cobbe et al. “Training Verifiers to Solve Math Word Problems”. Em: [arXiv preprint arXiv:2110.14168](#) (2021).
- [6] Dan Hendrycks et al. “Measuring Massive Multitask Language Understanding”. Em: [ICLR](#). 2021.
- [7] Jordan Hoffmann et al. “Training Compute-Optimal Large Language Models”. Em: [NeurIPS](#). Vol. 35. 2022.
- [8] Albert Q. Jiang et al. “Mistral 7B”. Em: [arXiv preprint arXiv:2310.06825](#) (2023).
- [9] Albert Q. Jiang et al. “Mixtral of Experts”. Em: [arXiv preprint arXiv:2401.04088](#) (2024).
- [10] Jared Kaplan et al. “Scaling Laws for Neural Language Models”. Em: [arXiv preprint arXiv:2001.08361](#) (2020).
- [11] Yaniv Leviathan, Matan Kalman e Yossi Matias. “Fast Inference from Transformers via Speculative Decoding”. Em: [ICML](#). 2023, pp. 19274–19286.
- [12] Alec Radford et al. [Improving Language Understanding by Generative Pre-Training](#). Rel. técn. OpenAI, 2018. URL: <https://openai.com/blog/language-unsupervised>.
- [13] Alec Radford et al. [Language Models are Unsupervised Multitask Learners](#). Rel. técn. OpenAI, 2019. URL: <https://openai.com/blog/better-language-models>.
- [14] Noam Shazeer. “Fast Transformer Decoding: One Write-Head is All You Need”. Em: [arXiv preprint arXiv:1911.02150](#) (2019).
- [15] Noam Shazeer. “GLU Variants Improve Transformer”. Em: [arXiv preprint arXiv:2002.05202](#) (2020).
- [16] Noam Shazeer et al. “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer”. Em: [ICLR](#). 2017.

Referências II

- [17] Stanford CME295 Teaching Staff. Lecture 2: Modern LLM Architectures. Stanford CME295: Large Language Models in Practice, Fall 2025. 2025. URL: <https://cme295.stanford.edu/slides/fall25-cme295-lecture2.pdf>.
- [18] Ashish Vaswani et al. “Attention is all you need”. Em: NeurIPS. Vol. 30. 2017.
- [19] Brandon T. Willard e Rémi Louf. “Efficient Guided Generation for Large Language Models”. Em: arXiv preprint arXiv:2307.09702 (2023).
- [20] Rowan Zellers et al. “HellaSwag: Can a Machine Really Finish Your Sentence?” Em: ACL. 2019, pp. 4791–4800.
- [21] Biao Zhang e Rico Sennrich. “Root Mean Square Layer Normalization”. Em: NeurIPS. Vol. 32. 2019.